In this exercise you will work with two **vision foundation models**—DINOv2 and OpenCLIP—and implement a **conditional flow matching** generative model on the CLEVR dataset. The three tasks are implemented as Jupyter notebooks that run on the TF pool cluster.

Submitting

What to submit. Upload a single `.zip` archive to ILIAS containing exactly the following files, preserving the directory structure shown below. **Do not include any PyTorch checkpoint files (`.pth`), dataset files, or any other large binary assets.**

```
submission.zip
  classification/
    classification_exercise.ipynb      (cleaned, no cell outputs)
    results/
      accuracies.json
      knn_sweep.json
      knn_accuracy_vs_k.png
      retrieval_chair_inside.png
      retrieval_chair_outside.png
  correspondences/
    correspondence_exercise.ipynb      (cleaned, no cell outputs)
    results/
      table1_pck_per_category.json
      table2_pck_by_dim.json
      viz_full_rgb.png
      viz_pca6_first3.png
      viz_pca6_last3.png
  flow_matching/
    flow_matching_exercise.ipynb       (cleaned, no cell outputs)
    results/
      flow_matching/
        accuracies.json
        mean_image_001.png  mean_image_100.png  mean_image_010.png
        mean_image_000.png  mean_image_011.png  mean_image_110.png
        mean_image_111.png  mean_image_101.png
      baseline/
        accuracies.json
        mean_image_001.png  mean_image_100.png  mean_image_010.png
        mean_image_000.png  mean_image_011.png  mean_image_110.png
        mean_image_111.png  mean_image_101.png
```

Cleaning notebooks. Before uploading, clear all cell outputs from each notebook. In JupyterLab: *Edit → Clear All Outputs*. Via command line:

```
jupyter nbconvert --ClearOutputPreprocessor.enabled=True \
    --to notebook --inplace classification_exercise.ipynb
```

Notebooks with cell outputs will not be graded.

Report. Write a short report (max 1 page, PDF) answering the discussion questions marked **Q:** throughout this sheet. Include the report as `report.pdf` at the top level of the zip.

Grading. Each sub-task is worth the number of points listed in the point summary. The total is **10 points**.

Setup

All exercises run on the TF pool cluster. Datasets and pre-trained models are available at:

`/project/dl2026s/lmbcvtst/data/`

Connect to a pool machine with a free GPU:

```
ssh username@login.informatik.uni-freiburg.de
ssh tfpoolXX       # prefer tfpool25-46 or tfpool51-63
nvidia-smi         # verify GPU is free
```

Start a screen session before launching Jupyter so training continues if you disconnect:

```
screen -S lab
jupyter notebook --no-browser --port 8888
# detach with Ctrl+A  D
```

Tunnel the Jupyter port to your local browser:

```
# on your local machine
ssh -N -L 8888:localhost:8888 username@login.informatik.uni-freiburg.de
```

Working on your own machine. If you prefer to work on your own hardware (e.g. a local GPU or Google Colab), you can download the required datasets and pre-trained models from Nextcloud:

https://nc.informatik.uni-freiburg.de/index.php/s/9pKHxKHDNYQAx7G

Download and extract the archive so that its contents mirror the directory structure expected by the notebooks (i.e. the same layout as `/project/dl2026s/lmbcvtst/data/` on the cluster), then adjust the `DATA_ROOT` variable at the top of each notebook accordingly.

1 Classification with DINOv2 and OpenCLIP

Open `classification/classification_exercise.ipynb`.

You will implement classifiers using frozen embeddings from two foundation models:

- **DINOv2** (`facebook/dinov2-base`, ViT-B/14): self-supervised; 768-dimensional CLS token.
- **OpenCLIP** (`ViT-B-16`, LAION-2B): trained on image-text pairs; 512-dimensional image and text embeddings.

The dataset is **SPair-71K**, containing images from 18 object categories. The `trn` split is used for training (index building) and the `test` split for evaluation.

1.1 kNN Classification (2 points)

Implement `KNNClassifier.fit()` and `KNNClassifier.predict()` in cell `cell_knn`. The class accepts a `backbone_type` parameter (`'dino'` or `'clip'`) and must: extract image embeddings from the chosen backbone using the provided `extract_features()` helper, store training embeddings and integer class labels, find the k nearest neighbours by L2 distance for each test image, and predict via majority voting.

The notebook runs both backbones at $k = 5$ and saves a sweep of $k \in \{1, \ldots, 10\}$ to `results/knn_accuracy_vs_k.png` and `results/knn_sweep.json`.

Q 1.1: Which backbone achieves higher accuracy and at which k? How do you explain the difference between DINOv2 and OpenCLIP for this task?

1.2 Zero-Shot Classification (2 points)

Implement `OpenCLIPZeroShotClassifier.fit()` and `.predict()` in cell `cell_zs`. The classifier must: build one text prompt per category in the form `"a photo of <category>"`, encode the prompts with the CLIP text encoder using `extract_features()`, encode test images with the CLIP image encoder, and predict the category with the highest cosine similarity to the image embedding.

Q 1.2: How does zero-shot accuracy compare to both kNN baselines? What does this tell you about the alignment between CLIP's image and text embedding spaces?

1.3 Text-to-Image Retrieval (1 point)

Implement `TextToImageRetriever.build_index()`, `.search()`, and `.visualize_results()` are already provided; you must complete `build_index()` and `search()` in cell `cell_ret`. The retriever must: index all test images by their CLIP embeddings, encode a free-form text query with the CLIP text encoder, and return the top-k most similar images by L2 distance.

The notebook runs two queries (`"chair inside"` and `"chair outside"`) and saves the result grids.

2 Semantic Correspondences with Patch Features

Open `correspondences/correspondence_exercise.ipynb`.

You will match **patch-level** features between image pairs from SPair-71K. Each backbone produces one embedding per patch: DINOv2 ViT-B/14 yields $16\times16 = 256$ patches; OpenCLIP ViT-B/16 yields $14\times14 = 196$ patches.

Correspondences are evaluated with **PCK** (Percentage of Correct Keypoints): a prediction is correct if it falls within $\alpha \cdot \max(\text{bbox\_width}, \text{bbox\_height})$ of the ground-truth target keypoint ($\alpha = 0.1$).

2.1 PCA for Patch Features (1 point)

Implement the `apply_pca(src_dict, trg_dict, n_components)` function in cell `ex-helpers`. The function must: concatenate all patch tensors from both dictionaries into a single NumPy array, fit `sklearn.decomposition.PCA(n_components, random_state=0)` on the combined array so both dictionaries share the same subspace, project each tensor with the fitted PCA, and return the projected tensors L2-normalised along the feature dimension.

In the same cell `ex-pca-fit`, also complete the PCA-6 fitting block for the `cat` and `dog` categories. Fit a 6-component PCA separately for DINOv2 and OpenCLIP, and compute per-component global min/-max bounds needed for consistent colour scaling. Store the results in `pca6_dino`, `pca6_clip`, `dino6_vmin`, `dino6_vmax`, `clip6_vmin`, `clip6_vmax`.

Once implemented, the notebook produces Table 1 (PCK per category), Table 2 (PCK vs. dimensionality from full down to PCA-16), a raw-RGB correspondence visualisation, and two PCA-6 colour-overlay visualisations.

Q 2.1: At which PCA dimensionality does the PCK accuracy start to drop significantly? Do semantically similar regions (e.g. cat legs, dog legs) in different images receive similar colours in the PCA-6 visualisation?

3 Conditional Flow Matching on CLEVR

Open `flow_matching/flow_matching_exercise.ipynb`.

You will implement a **conditional flow matching** generative model and compare it to a direct regression baseline on the CLEVR dataset. Both models use a shared `ContextUnet` backbone conditioned on three attributes: *shape* (3 classes), *colour* (3-d RGB), and *size* (scalar). Generated samples are evaluated by a pre-trained linear probe that checks whether the rendered image matches the requested attributes, separately for *seen* (training) and *unseen* (test) attribute combinations.

Complete the `training_model` function (cell `a2a15059`) shared by both models: zero the gradients, compute the loss via `model(x, cl)`, backpropagate, and update the parameters.

3.1 Direct Regression Baseline (2 points)

Implement `Regression.forward()` and `Regression.sample()` in cell `79c69150`.

In **forward**: pass an all-ones image and a time tensor of ones ($t = 1$) to the network, and return the MSE between the prediction and the target image x_0. In **sample**: perform a single forward pass from the all-ones image with $t = 1$ and return the prediction directly—no iterative integration is needed.

Hint: A well-trained regression model should reach a training loss below 0.001.

3.2 Flow Matching (2 points)

Implement `FlowMatching.forward()` and `FlowMatching.sample()` in cell `647184d7`.

In **forward**: sample noise $\epsilon \sim \mathcal{N}(0, I)$ and time $t \sim \mathcal{U}(0, 1)$, form the interpolated image $x_t = (1-t)\, x_0 + t\, \epsilon$, and return the MSE between the predicted velocity $\hat{v}_\theta(x_t, c, t)$ and the target velocity $v^* = \epsilon - x_0$. In **sample**: start from pure Gaussian noise at $t = 1$ and integrate backward to $t = 0$ using Euler steps of size $\Delta t = 1/n_T$:

$$x_{t-\Delta t} = x_t - \hat{v}_\theta(x_t, c, t) \cdot \Delta t.$$

Hint: A well-trained flow matching model should reach a training loss below 0.03.

Q 3.2: Compare mean accuracy of flow matching vs. the regression baseline on seen and unseen configurations. When does flow matching perform better?

Point Summary

Exercise	Sub-task	Points
1 – Classification	1.1 kNN classification	2
	1.2 Zero-shot classification	2
	1.3 Text-to-image retrieval	1
2 – Correspondences	2.1 PCA for patch features	1
3 – Flow Matching	3.1 Regression baseline	2
	3.2 Flow matching	2
	Total	**10**